Details about regression in the R package Kostra

Øyvind Langsrud

October 13, 2016

Contents

1 Advanced use of lm **1**

1.1 Multiple responses 1

1.2 Prediction error 2

1.3 Robust variance 2

1 Advanced use of lm

1.1 Multiple responses

lm with multiple responses is problematic and this can be illustrated by running the following code.

```
1 y <- cbind(1:6,c(3,3,4,4,5,5))
2 x <- c(1,1,1,2,2,2)
3 summary( lm(y[,2] ~ x) )
4 predict( lm(y[,2] ~ x) ,se.fit=TRUE)
5 rstudent(lm(y[,2] ~ x) )
6 summary( lm(y ~ x) )
7 predict( lm(y ~ x), se.fit=TRUE)
8 rstudent(lm(y ~ x) )
```

Here y has two variables, but in lines 3-5 only the second variable is used.

One would expect the results of these lines to be replicated when both variables are input, but this is only the case for line 6. Line 7 results in the same fit as line 4, but the other output are missing. Line 8 produce strange results. The values are different and seem to be totally wrong.

The result of lm with multiple responses is a so-called mlm-object and consists of

several lm-objects. Running built-in functions on mlm-objects are problematic and two alternatives are

1. Split mlm-objects into separate lm-objects and run the function separately on each lm-object.
2. Write own functions based on matrix algebra.

For the function `LmImpute` we have chosen to rely on the available built-in functions in R and functions according to alternative 1 is therefore written (`Mlm2lm, MlmFunction`). We make use of the code posted by Brian Ripley at `http://grokbase.com/t/r/r-help/042s54v7py/r-structure-of-mlm-objects`.

1.2 Prediction error

The function `predict` (`predict.lm`) can be used to calculate prediction intervals, but corresponding standard errors are not in output. A generalized version, `SePredict` were this is included is therefore made. In addition a multiple response variant (`MlmSePredict`) is written.

1.3 Robust variance

A variant of robust variance estimation which is implemented in SSB's application, "struktur", is also implemented in `LmImpute`. In order to make such calculations one needs to find the weights underlying the estimates.

However, for a typical use of `LmImpute` only a small fraction of the observations are imputed and this is far from a situation with a small sampling fraction. The "negligible" component of the variance becomes instead the most important component. Robust variance estimation is also implemented in the r-package CalibrateSSB and according to the accompanying paper (Langsrud, 2016) the robust variance estimate can be written as

$$\sum w_i(w_i - 1)e_i^2 \tag{1}$$

were the e_i's are adjusted residuals (using $(1 - h_{ii})$ as divisor). This estimate is composed of two components

$$\sum(w_i - 1)^2 e_i^2 + \sum(w_i - 1)e_i^2 \tag{2}$$

were the second component function as an estimate of the variance outside the sample. This is the component that is negligible when the sampling fraction is small. When this component is important it is important how we estimate the unknown variance. It is not obvious that we should use a weighted

sum of the squared residuals using exactly the same weights as is used for imputation. The values of e_i can vary a lot and negative values of $(w_i - 1)$ can lead to negative variance estimates. To calculate robust variance in `LmImpute` the second component of (2) is modified (the first is not). To explain this modification we let $a_i = (w_i - 1)$. Negative values of a_i are set to zero and the remaining values are scaled by a constant so that the sum is unchanged. This is done in the function `RobustVar`. Note that in the r program w plays the role as $w - 1$.

The code below calculates robust variances using both the r-package Kostra (`LmImpute` via `ImputeRegression`) and the r-package CalibrateSSB. The data is taken from the example code of `ImputeRegression`.

```
library(CalibrateSSB)# version 1.0
library(Kostra)      # version 0.24
rateData <- KostraData("rateData")
d <- data.table(rateData$data[, c(16, 5, 19)])
d <- d[is.finite(Ny.kostragruppe), ]
d[Ny.kostragruppe>13,Ny.kostragruppe := 13]
a <- ImputeRegression(d,limitModel = 50, limitIterate = 50)
z <- as.data.frame(a$micro)
z$R <- as.numeric(is.finite(z$y))
N <-  dim(z)[1]
b <- CalibrateSSB(z,calmodel="~ x", partition="strata",
                  y="y",leverageOutput = TRUE)
bWide <- WideFromCalibrate(b,rep(1,N),1:N,subSet=z$strata)
p <- PanelEstimation(bWide,"y",leveragePower = 1)
print(a)
for(i in 1:12) cat(names(p)[i],p[[i]]$estimates,
                   sqrt(p[[i]]$varEstimates),"\n")
```

The result is that standard error estimates are identical except for strata number 13. In that case two weights are below one and the modification described above apply.

In `LmImpute` the weights are calculated by a generalized inverse similar to the description in Langsrud (2016). These calculations are done in the function `PredictSum` where the weights are found by multiplying

```
t(inv(x * sqrt_w)) * sqrt_w
```
(3)

with computed totals. Here `sqrt_w` is the square root of the weights used in the regression model (`(weights(m))`, `x` is the model matrix (`model.matrix(m)`) and `inv` is set to be the generalized inverse (`ginv`) in the r-package MASS.